10 — Schemas & Namespacing in PostgreSQL

Table of Contents

Learning Objectives

After completing this section you will be able to:

- Create and organize objects using schemas
- Configure `search_path` for correct object resolution
- Design permission models using schema-level grants
- Implement multi-tenant applications using schema-per-tenant
- Avoid common `search_path` injection vulnerabilities

What is a Schema?

A **schema** is a namespace within a database that groups related objects (tables, views, functions, sequences, types, etc.).

```
PostgreSQL Object Hierarchy:
┌──────────────────────────────────────────────────────────────┐
│  Cluster (PostgreSQL server)                                 │
│  ├── Database: myapp                                         │
│  │   ├── Schema: public    (default)                         │
│  │   │   ├── Table: users                                    │
│  │   │   ├── Table: orders                                   │
│  │   │   └── Function: calculate_total()                     │
│  │   ├── Schema: admin                                       │
│  │   │   ├── Table: audit_log                                │
│  │   │   └── View: user_summary                              │
│  │   ├── Schema: billing                                     │
│  │   │   ├── Table: invoices                                 │
```

```
│   │   └── Table: payments                                    │
│   └── Schema: pg_catalog  (system)                           │
└── Database: analytics                                        │
    └── Schema: public                                         │
└──────────────────────────────────────────────────────────────┘

Fully qualified name:  database.schema.object
  OR (within current database): schema.object
```

Key facts:

- Objects in different schemas can have the same name without conflict
- Schemas are per-database — you cannot reference objects in other databases without `dblink` or foreign data wrappers
- Every database has a `public` schema by default
- Every database has `pg_catalog` and `information_schema` schemas (system)

The search_path

`search_path` is a list of schemas that PostgreSQL searches when you reference an unqualified object name.

```sql
-- View current search_path
SHOW search_path;
-- "$user", public    ← default

-- "$user" resolves to a schema with the same name as the current user
-- (if it exists; otherwise it is ignored)

-- Set search_path for current session
SET search_path = public, billing, admin;

-- Set search_path permanently for a user
ALTER USER app_user SET search_path = app, public;

-- Set search_path permanently for a database
ALTER DATABASE myapp SET search_path = app, public;

-- Set search_path for a function (recommended for security)
CREATE FUNCTION foo() RETURNS void
LANGUAGE SQL
SET search_path = myschema, pg_catalog  -- pin the path
AS $$ SELECT 1 $$;
```

search_path resolution example

```
search_path = 'billing, public'

SELECT * FROM invoices;
  1. Look for billing.invoices   → found! Use it.
```

```
SELECT * FROM users;
  1. Look for billing.users      → not found
  2. Look for public.users       → found! Use it.

SELECT * FROM nonexistent;
  1. Look for billing.nonexistent → not found
  2. Look for public.nonexistent  → not found
  → ERROR: relation "nonexistent" does not exist
```

Creating and Managing Schemas

```sql
-- Create schema
CREATE SCHEMA IF NOT EXISTS billing;

-- Create schema with owner
CREATE SCHEMA billing AUTHORIZATION billing_role;

-- Create schema and objects in one statement
CREATE SCHEMA app
    CREATE TABLE users (id SERIAL, email TEXT)
    CREATE VIEW active_users AS SELECT * FROM users;

-- Drop schema
DROP SCHEMA billing;                    -- fails if schema is not empty
DROP SCHEMA billing CASCADE;            -- drops all contained objects
DROP SCHEMA billing RESTRICT;           -- same as DROP SCHEMA (default)

-- Rename schema
ALTER SCHEMA billing RENAME TO invoicing;

-- Change owner
ALTER SCHEMA billing OWNER TO billing_role;

-- List all schemas
SELECT schema_name, schema_owner
FROM information_schema.schemata
ORDER BY schema_name;

-- Or:
\dn    -- in psql
SELECT nspname, nspowner::regrole FROM pg_namespace;
```

Schema-Level Permissions

```sql
-- Schema privileges:
--   USAGE:  ability to reference objects in the schema (needed to use tables, etc.)
--   CREATE: ability to create objects in the schema
```

```sql
-- Grant usage (allows querying objects if table-level perms also granted)
GRANT USAGE ON SCHEMA billing TO app_user;

-- Grant create (allows creating new objects)
GRANT CREATE ON SCHEMA billing TO billing_admin;

-- Revoke
REVOKE CREATE ON SCHEMA billing FROM billing_admin;

-- Common pattern: read-only user
GRANT USAGE ON SCHEMA public TO readonly_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly_user;
-- Future tables:
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT ON TABLES TO readonly_user;

-- Common pattern: app user (read/write, no DDL)
GRANT USAGE ON SCHEMA public TO app_user;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_user;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO app_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_user;
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT USAGE ON SEQUENCES TO app_user;

-- Public schema permissions (PostgreSQL 15+ changed defaults)
-- In PG 15+: public schema no longer world-writable by default
-- You must explicitly grant CREATE on public schema
GRANT CREATE ON SCHEMA public TO app_user;
```

Default Schemas in PostgreSQL

```
Schema          Purpose
─────────────────────────────────────────────────────────────
public          Default schema for user objects
pg_catalog      System catalog tables (pg_class, pg_attribute, etc.)
information_schema  SQL standard views of catalog data
pg_toast        TOAST storage (not directly accessible)
pg_temp_N       Per-session temporary tables
pg_toast_temp_N TOAST storage for temp tables
```

```sql
-- System catalog examples
SELECT relname, relkind
FROM pg_catalog.pg_class
WHERE relnamespace = 'public'::regnamespace
  AND relkind IN ('r', 'v', 'i')  -- tables, views, indexes
ORDER BY relname;

-- information_schema: portable SQL standard views
```

```sql
SELECT table_name, table_type
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name = 'users'
ORDER BY ordinal_position;
```

Cross-Schema Queries

```sql
-- Fully qualified names
SELECT * FROM billing.invoices;
SELECT * FROM public.users u JOIN billing.invoices i ON u.user_id = i.user_id;

-- Cross-schema view
CREATE VIEW public.invoice_with_customer AS
SELECT
    i.invoice_id,
    i.amount,
    u.email AS customer_email,
    u.full_name
FROM billing.invoices i
JOIN public.users u ON u.user_id = i.user_id;

-- Cross-database: NOT directly possible
-- Use foreign data wrapper (FDW) for cross-database access
CREATE EXTENSION IF NOT EXISTS postgres_fdw;

CREATE SERVER remote_db
    FOREIGN DATA WRAPPER postgres_fdw
    OPTIONS (host 'remote.server.com', dbname 'analytics');

CREATE USER MAPPING FOR app_user
    SERVER remote_db
    OPTIONS (user 'remote_user', password 'secret');

CREATE FOREIGN TABLE remote_events (
    event_id    BIGINT,
    event_type  TEXT,
    occurred_at TIMESTAMPTZ
) SERVER remote_db OPTIONS (schema_name 'public', table_name 'events');

SELECT * FROM remote_events WHERE occurred_at > now() - INTERVAL '1 day';
```

Schema Design Patterns

Pattern 1: Functional schemas (recommended for medium-large apps)

```
Database: myapp
├── public          — shared/core tables (users, roles, etc.)
├── billing         — invoices, payments, subscriptions
├── catalog         — products, categories, inventory
├── orders          — orders, order_items, shipping
├── notifications   — email_queue, push_queue, templates
├── analytics       — aggregated tables, materialized views
└── admin           — audit_log, config, admin-only views
```

Pattern 2: Versioned API schemas

```
Database: myapp
├── api_v1          — views/functions exposing v1 API shape
├── api_v2          — views/functions exposing v2 API shape
└── internal        — actual tables (never exposed directly)
```

Pattern 3: ETL/Data Warehouse schemas

```
Database: warehouse
├── raw             — raw ingested data (no transformations)
├── staging         — cleaned/validated data
├── core            — normalized/modeled facts and dimensions
├── marts           — denormalized reporting tables
└── reports         — final views for BI tools
```

Multi-Tenant Schema Patterns

Schema-per-tenant model

```
-- Create schema for each tenant
CREATE OR REPLACE FUNCTION create_tenant_schema(p_tenant_id TEXT)
RETURNS void LANGUAGE plpgsql AS $$
BEGIN
    EXECUTE format('CREATE SCHEMA IF NOT EXISTS tenant_%s', p_tenant_id);

    EXECUTE format($$
        CREATE TABLE tenant_%s.users (
            user_id     BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            email       TEXT   NOT NULL UNIQUE,
            full_name   TEXT   NOT NULL,
            created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
        )
    $$, p_tenant_id);

    EXECUTE format($$
        CREATE TABLE tenant_%s.orders (
            order_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
```

```
            user_id     BIGINT NOT NULL REFERENCES tenant_%s.users(user_id),
            total       NUMERIC(12,4) NOT NULL,
            created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
        )
    $$, p_tenant_id, p_tenant_id);

    -- Grant to tenant-specific role
    EXECUTE format('GRANT USAGE ON SCHEMA tenant_%s TO tenant_%s_role', p_tenant_id,
p_tenant_id);
    EXECUTE format('GRANT ALL ON ALL TABLES IN SCHEMA tenant_%s TO tenant_%s_role',
p_tenant_id, p_tenant_id);
END;
$$;

-- Create tenant
SELECT create_tenant_schema('acme');
SELECT create_tenant_schema('globex');

-- Set tenant context at connection time
ALTER USER acme_app SET search_path = tenant_acme, public;

-- Application queries work without schema prefix
SET search_path = tenant_acme, public;
SELECT * FROM users;   -- actually selects tenant_acme.users
```

Row-Level Security (shared schema, alternative to schema-per-tenant)

```
-- See ../14_Security/ for RLS-based multi-tenancy
-- schema-per-tenant vs RLS trade-offs covered in
06_Database_Design/07_schema_design_saas.md
```

Extensions and Schemas

```
-- Install extension in a specific schema
CREATE EXTENSION IF NOT EXISTS pg_trgm SCHEMA extensions;
CREATE EXTENSION IF NOT EXISTS "uuid-ossp" SCHEMA extensions;

-- Add extensions schema to search_path
ALTER DATABASE myapp SET search_path = public, extensions;

-- View installed extensions
SELECT extname, extversion, nspname AS schema
FROM pg_extension
JOIN pg_namespace ON pg_namespace.oid = extnamespace;

-- Relocate an extension to a different schema (if supported)
ALTER EXTENSION pg_trgm SET SCHEMA public;
```

SQL Examples

Complete schema setup for a SaaS application

```sql
-- Create schemas
CREATE SCHEMA IF NOT EXISTS core;
CREATE SCHEMA IF NOT EXISTS billing;
CREATE SCHEMA IF NOT EXISTS analytics;

-- Roles
CREATE ROLE app_readonly;
CREATE ROLE app_readwrite;
CREATE ROLE billing_admin;

-- Core schema: shared user data
SET search_path = core;

CREATE TABLE users (
    user_id     BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email       TEXT    NOT NULL UNIQUE,
    full_name   TEXT    NOT NULL,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE tenants (
    tenant_id   BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name        TEXT    NOT NULL UNIQUE,
    slug        TEXT    NOT NULL UNIQUE CHECK (slug ~ '^[a-z0-9\-]+$'),
    is_active   BOOLEAN NOT NULL DEFAULT TRUE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE tenant_users (
    tenant_id   BIGINT NOT NULL REFERENCES tenants(tenant_id),
    user_id     BIGINT NOT NULL REFERENCES users(user_id),
    role        TEXT   NOT NULL DEFAULT 'member'
                    CHECK (role IN ('owner','admin','member')),
    joined_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    PRIMARY KEY (tenant_id, user_id)
);

-- Billing schema: financial data
SET search_path = billing;

CREATE TABLE subscriptions (
    sub_id      BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tenant_id   BIGINT  NOT NULL REFERENCES core.tenants(tenant_id),
    plan        TEXT    NOT NULL CHECK (plan IN ('starter','pro','enterprise')),
    status      TEXT    NOT NULL DEFAULT 'active'
                    CHECK (status IN ('active','cancelled','past_due')),
    started_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
```

```
    ends_at     TIMESTAMPTZ
);

CREATE TABLE invoices (
    invoice_id  BIGINT  GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    tenant_id   BIGINT  NOT NULL REFERENCES core.tenants(tenant_id),
    amount      NUMERIC(12,4) NOT NULL CHECK (amount > 0),
    status      TEXT    NOT NULL DEFAULT 'unpaid'
                    CHECK (status IN ('unpaid','paid','void')),
    issued_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    due_at      TIMESTAMPTZ NOT NULL DEFAULT now() + INTERVAL '30 days'
);

-- Analytics schema: reporting
SET search_path = analytics;

CREATE MATERIALIZED VIEW tenant_summary AS
SELECT
    t.tenant_id,
    t.name AS tenant_name,
    t.slug,
    count(DISTINCT tu.user_id) AS user_count,
    s.plan AS subscription_plan,
    s.status AS subscription_status,
    sum(i.amount) FILTER (WHERE i.status = 'paid') AS total_paid
FROM core.tenants t
LEFT JOIN core.tenant_users tu ON tu.tenant_id = t.tenant_id
LEFT JOIN billing.subscriptions s ON s.tenant_id = t.tenant_id AND s.status =
'active'
LEFT JOIN billing.invoices i ON i.tenant_id = t.tenant_id
GROUP BY t.tenant_id, t.name, t.slug, s.plan, s.status;

CREATE UNIQUE INDEX ON analytics.tenant_summary (tenant_id);

-- Grant permissions
GRANT USAGE ON SCHEMA core TO app_readonly, app_readwrite;
GRANT USAGE ON SCHEMA billing TO app_readwrite, billing_admin;
GRANT USAGE ON SCHEMA analytics TO app_readonly, app_readwrite;

GRANT SELECT ON ALL TABLES IN SCHEMA core TO app_readonly;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA core TO app_readwrite;

GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA billing TO
billing_admin;

-- Reset search_path
RESET search_path;
```

Common Mistakes

1. **Relying on `public` for everything** — As applications grow, everything in `public` becomes hard to manage. Use schemas for functional grouping from day one.
2. **Not setting `search_path` for functions (security risk)**

```
-- RISKY: if a malicious user creates a table/function named
'sensitive_function'
-- in their schema and it's earlier in search_path...
CREATE FUNCTION risky() RETURNS void AS $$ SELECT sensitive_function(); $$
LANGUAGE sql;

-- SAFE: pin the path
CREATE FUNCTION safe() RETURNS void
LANGUAGE sql SET search_path = myschema, pg_catalog
AS $$ SELECT myschema.sensitive_function(); $$;
```

3. **Forgetting to grant USAGE on schema in addition to table privileges**

```
-- This is insufficient:
GRANT SELECT ON billing.invoices TO app_user;
-- ERROR: permission denied for schema billing

-- Must also:
GRANT USAGE ON SCHEMA billing TO app_user;
GRANT SELECT ON billing.invoices TO app_user;
```

4. **Not using `ALTER DEFAULT PRIVILEGES` for future objects**

```
-- Granting on existing tables but not future ones:
GRANT SELECT ON ALL TABLES IN SCHEMA public TO readonly;
-- New tables created later won't be included!

-- Fix: add default privileges
ALTER DEFAULT PRIVILEGES IN SCHEMA public
    GRANT SELECT ON TABLES TO readonly;
```

5. **Using schema names with special characters or uppercasing**

```
CREATE SCHEMA "MySchema";   -- requires quotes everywhere!
SELECT * FROM "MySchema".users;  -- annoying

-- Better: lowercase, no special chars
CREATE SCHEMA my_schema;
SELECT * FROM my_schema.users;
```

Best Practices

1. **Use schemas for functional grouping** of related tables/views/functions.

2. **Never put application objects in `pg_catalog`** — it's for system objects.
3. **Set `search_path` explicitly** for application roles; don't rely on defaults.
4. **Always pin `search_path` in functions** to prevent injection attacks.
5. **Use `ALTER DEFAULT PRIVILEGES`** to ensure new objects inherit the right permissions.
6. **Keep `public` schema minimal** — use it for truly shared objects or not at all.
7. **For extensions**, create a dedicated `extensions` schema and add it to `search_path`.
8. **Document schema purposes** in `COMMENT ON SCHEMA`: `COMMENT ON SCHEMA billing IS 'Financial data: invoices, payments, subscriptions'`.

Performance Considerations

- Schema separation has **zero performance impact** — schemas are purely a namespace/permission mechanism, not a storage boundary.
- **Schema-per-tenant** approach means: more schemas → more objects in `pg_class` → slightly slower catalog queries (negligible at < 10,000 tenants).
- For **very large multi-tenant systems** (> 10,000 tenants), prefer row-level security (shared schema) over schema-per-tenant to avoid catalog bloat.
- `search_path` resolution adds a tiny overhead per query (catalog lookup) but is cached — negligible in practice.
- **Function `search_path` pinning**: functions with a pinned `search_path` can be `IMMUTABLE` and may be inlined by the query planner.

Interview Questions & Answers

Q1: What is a schema in PostgreSQL and how is it different from a database?

A: A schema is a namespace within a database. A database is a completely separate collection of schemas, tables, and other objects — cross-database queries are not directly possible without FDW. Multiple schemas in one database can share data through joins. Databases are isolated. Use schemas for logical grouping within a database; use separate databases for completely isolated applications or environments.

Q2: What is `search_path` and why does it matter?

A: `search_path` is the ordered list of schemas PostgreSQL searches when resolving unqualified object names. If `search_path = 'billing, public'` and you write `SELECT * FROM invoices`, PostgreSQL looks for `billing.invoices` first, then `public.invoices`. It matters for security (schema injection attacks), portability (moving objects between schemas), and convenience (not having to qualify every name).

Q3: What is a `search_path` injection attack?

A: If a function doesn't pin its `search_path` and a malicious user creates an object with the same name in a schema that appears earlier in the search path, the function might call that malicious object instead of the intended one. The fix is to always set `search_path` in function definitions: `CREATE FUNCTION ... SET search_path = specific_schema, pg_catalog`.

Q4: How do schema permissions work — what is the difference between USAGE and CREATE?

A: `USAGE` on a schema allows users to reference objects within it (tables, functions, etc.) — they still need object-level permissions (SELECT, INSERT, etc.). `CREATE` allows users to create new objects (tables, functions, views) in the schema. Granting table SELECT without schema USAGE results in "permission denied for schema."

Q5: What is `ALTER DEFAULT PRIVILEGES` and when is it needed?

A: It sets permissions that are automatically applied to future objects created in a schema. Without it, every new table/function/view created by user A requires explicit grants to user B. With `ALTER DEFAULT PRIVILEGES IN SCHEMA public GRANT SELECT ON TABLES TO readonly`, all future tables automatically have SELECT granted to readonly.

Q6: How does schema-per-tenant differ from shared-schema multi-tenancy?

A: Schema-per-tenant: each tenant has their own schema with identical table structures. Strong isolation, easy per-tenant backup/restore, but catalog bloat at scale. Shared-schema: all tenants share the same tables with a `tenant_id` column; Row-Level Security controls access. Better performance at scale, simpler schema management, but weaker isolation boundary.

Q7: What happens to schema objects when you `DROP SCHEMA ... CASCADE`?

A: All objects in the schema are dropped: tables (and their data!), views, functions, sequences, types, etc. This is irreversible. Always test schema drops in a non-production environment first and ensure you have a backup.

Q8: What is the difference between `pg_catalog` and `information_schema`?

A: `pg_catalog` contains PostgreSQL-specific system tables (native format, most detailed, fastest). `information_schema` contains SQL-standard views that present catalog information in a portable format across databases. Use `information_schema` for portable scripts; use `pg_catalog` for PostgreSQL-specific tools and maximum performance.

Exercises with Solutions

Exercise 1

Create a multi-schema application with separate schemas for `app` (user-facing) and `audit` (logging). Set up appropriate permissions for a read-write app role and an audit reader role.

Solution:

```
CREATE SCHEMA app;
CREATE SCHEMA audit;

CREATE ROLE app_rw;
CREATE ROLE audit_reader;

-- App schema
CREATE TABLE app.users (
    user_id     BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email       TEXT NOT NULL UNIQUE,
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Audit schema
CREATE TABLE audit.user_changes (
    change_id   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    table_name  TEXT NOT NULL,
    operation   TEXT NOT NULL,
```

```
    old_data    JSONB,
    new_data    JSONB,
    changed_by  TEXT NOT NULL DEFAULT current_user,
    changed_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Permissions
GRANT USAGE ON SCHEMA app TO app_rw;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA app TO app_rw;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA app TO app_rw;

GRANT USAGE ON SCHEMA audit TO app_rw;
GRANT INSERT ON audit.user_changes TO app_rw;

GRANT USAGE ON SCHEMA audit TO audit_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA audit TO audit_reader;

-- Future objects
ALTER DEFAULT PRIVILEGES IN SCHEMA app
    GRANT SELECT, INSERT, UPDATE, DELETE ON TABLES TO app_rw;
ALTER DEFAULT PRIVILEGES IN SCHEMA audit
    GRANT SELECT ON TABLES TO audit_reader;
```

Exercise 2

Write a stored procedure that dynamically creates a new tenant schema with a standardized set of tables.

Solution:

```
CREATE OR REPLACE PROCEDURE provision_tenant(p_tenant_slug TEXT)
LANGUAGE plpgsql AS $$
DECLARE
    v_schema TEXT := format('tenant_%s', p_tenant_slug);
BEGIN
    -- Validate slug
    IF p_tenant_slug !~ '^[a-z][a-z0-9_]{1,62}$' THEN
        RAISE EXCEPTION 'Invalid tenant slug: %', p_tenant_slug;
    END IF;

    -- Create schema
    EXECUTE format('CREATE SCHEMA IF NOT EXISTS %I', v_schema);

    -- Standard tables
    EXECUTE format($$
        CREATE TABLE %I.settings (
            key     TEXT PRIMARY KEY,
            value   JSONB NOT NULL,
            updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
        )
    $$, v_schema);

    EXECUTE format($$
```

```
        CREATE TABLE %I.users (
            user_id    BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
            email      TEXT NOT NULL UNIQUE,
            role       TEXT NOT NULL DEFAULT 'member',
            created_at TIMESTAMPTZ NOT NULL DEFAULT now()
        )
    $$, v_schema);

    RAISE NOTICE 'Tenant schema % created successfully', v_schema;
END;
$$;

CALL provision_tenant('acme_corp');
CALL provision_tenant('beta_startup');
```

Cross-References

- `08_constraints.md` — constraint naming across schemas
- `../14_Security/` — schema permissions and Row-Level Security
- `../06_Database_Design/07_schema_design_saas.md` — multi-tenant patterns in depth
- `../12_Production_PostgreSQL/` — schema maintenance and monitoring